

Module 3 | Numerical Quadrature



The past two chapters have developed a suite of tools for polynomial interpolation and approximation. Now we are going to apply these tools toward the approximation of definite integrals.

Most definite integrals are difficult or impossible to evaluate exactly. For example, one of the important integral in probability and statistics is the *error function*

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt.$$

which has no closed form formula for the indefinite integral. Then how does a computer evaluate values for this function?

Integrals also appear intrinsically in approximation theory. For instance, in least squares approximation we repeatedly encounter inner products of the form

$$\langle f, \varphi_j \rangle = \int_a^b f(x) \varphi_j(x) w(x) dx,$$

so even the task of computing a best approximation can reduce to evaluating definite integrals.

So this chapter will be about developing algorithms that approximate such integrals quickly and accurately. The difficulties associated with symbolic integration are so serious that you have already seen some numerical methods for approximating definite integrals in calculus (Riemann Sums). We'll recap some of these in a bit more detail than you might recall, and introduce a few new ideas along the way.

§3.1 Quadrature rules

Since we want to sound more fancy than a calculus course, we will use the exotic new term **quadrature** to refer to numerically approximating definite integrals. The term “quadrature” in math historically refers to finding the area of a geometric figure in the plane by dividing it into shapes of known area (e.g., squares, rectangles or triangles), and many of the numerical integration methods from calculus do just that.

A large class of quadrature rules is built from interpolation/approximation: choose a simple approximant p to f (typically a piecewise polynomial), and then set

$$\int_a^b f(x) dx \approx \int_a^b p(x) dx,$$

reducing quadrature to (i) constructing p from point samples and (ii) integrating p exactly.

3.1.1 Riemann sums

The oldest and simplest examples come from Riemann sums. Fix a partition

$$a = x_0 < x_1 < \cdots < x_n = b, \quad h_i := x_{i+1} - x_i.$$

Approximating f on each interval $[x_i, x_{i+1}]$ by a constant yields the rectangle rules:

$$Q_L(f) = \sum_{i=0}^{n-1} h_i f(x_i), \quad Q_R(f) = \sum_{i=0}^{n-1} h_i f(x_{i+1}), \quad Q_M(f) = \sum_{i=0}^{n-1} h_i f\left(\frac{x_i + x_{i+1}}{2}\right).$$

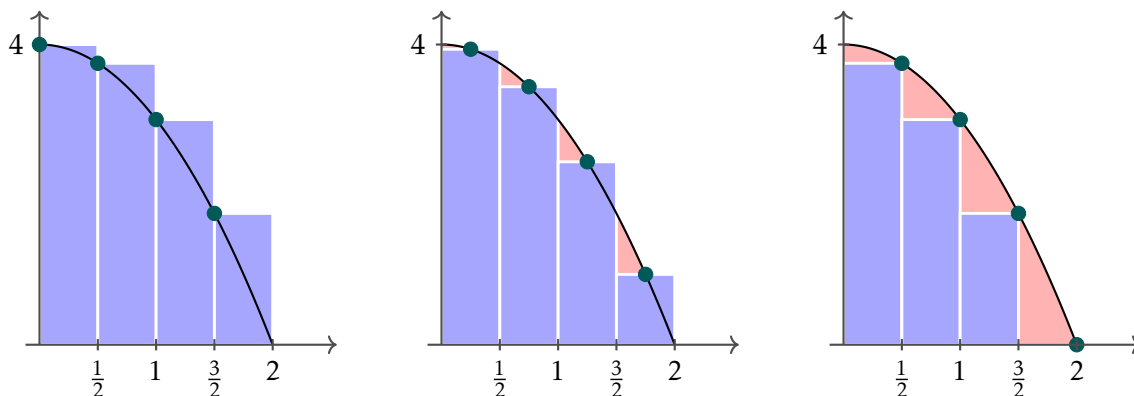


Figure 3.1: Left Riemann sum, midpoint Riemann sum, and right Riemann sum.

While writing the formula for the Riemann sums, you might have observed that each of the methods approximates the definite integral by a weighted average of the function values at an ordered set of points $\{x_0, x_1, \dots, x_n\}$:

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i)$$

where the coefficients w_i are the weights. Often (but not always), quadrature rules use only points x_i in the interval $[a, b]$. All quadrature methods are essentially about choosing the x_i and w_i in an efficient way.

■ Question 36.

□

Suppose we wish to approximate $I(f) := \int_a^b f(x) dx$ using a uniform partition $x_i = a + ih$ with $h = (b - a)/n$.

(a) If $f \in C^1([a, b])$, then prove that

$$|I(f) - Q_L(f)| \leq \frac{b-a}{2} \|f'\|_\infty h, \quad |I(f) - Q_R(f)| \leq \frac{b-a}{2} \|f'\|_\infty h.$$

(b) If $f \in C^2([a, b])$, then prove that

$$|I(f) - Q_M(f)| \leq \frac{b-a}{24} \|f''\|_\infty h^2.$$

3.1.2 Trapezoidal Rule

You may have heard of the trapezoidal rule for approximating a definite integral.

$$Q_T(f) := \sum_{i=0}^{n-1} \frac{h_i}{2} (f(x_i) + f(x_{i+1})).$$

In the uniform-mesh case $h_i \equiv h$, this becomes

$$Q_T(f) = \frac{h}{2} \left(f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right).$$

Here is a picture to describe the process:

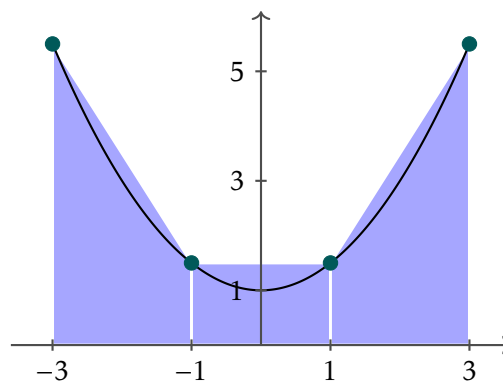


Figure 3.2: Trapezoid rule

Our knowledge about polynomial interpolant gives us a new way to think about this rule. Notice that the trapezoid rule essentially integrates the linear f -interpolation of the endpoints over each subinterval (i.e., it integrates the *linear spline*). It may not have occurred to you that the other three rules (left-endpoint, right-endpoint, and midpoint) can be considered as integration of different 0-degree interpolations (i.e., constants) on each subinterval (so they integrate a kind of pseudo-spline where the endpoints don't necessarily connect).

Note: On a uniform mesh the trapezoidal rule is the average of the left and right rectangle rules:

$$Q_T(f) = \frac{1}{2} (Q_L(f) + Q_R(f)).$$

■ Question 37.

Let $f \in C^2([a, b])$ and $x_i = a + ih$ with $h = (b - a)/n$. Then prove that

$$|I(f) - Q_T(f)| \leq \frac{b-a}{12} \|f''\|_{\infty} h^2.$$

What happens if we consider an higher degree spline or a different interpolant? As we will see in [lab assignment 3.5](#), this indeed reduces the error – sometimes by an even greater margin than we might expect.

3.1.3 Newton-Cotes Quadrature

Newton-Cotes quadrature is based on choosing **Lagrange interpolation with equally spaced points**. In that sense, the trapezoidal rule is also a Newton-Cotes quadrature rule that uses a two-point interpolant on each subinterval. Here are some other options.

Simpson's Rule

Suppose we wish to use three-point interpolants on each subinterval, i.e., quadratic splines.

We will start simple: consider a subinterval of the form $[x_0, x_2]$ with midpoint $x_1 = \frac{x_0 + x_2}{2}$.

Using the Lagrange basis,

$$P_2(x) = f(x_0) \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + f(x_2) \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

Let's integrate our interpolant to find an approximation of the integral:

$$\int_{x_0}^{x_2} f(x) dx \approx f(x_0) \int_{x_0}^{x_2} \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} dx + f(x_1) \int_{x_0}^{x_2} \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} dx + f(x_2) \int_{x_0}^{x_2} \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} dx$$

Assume we have a uniform partition where the width of the subinterval is h . In other words, $x_1 - x_0 = x_2 - x_1 = \frac{h}{2}$. Then, after some change of variable calculation, we get

$$\int_{x_0}^{x_2} f(x) dx \approx \frac{h}{6} (f(x_0) + 4f(x_1) + f(x_2)) =: Q_{SR}f$$

The last formula is known as **Simpson's Rule** for approximating definite integrals.

Note: In this formula,

$$w_0 = \frac{h}{6}, \quad w_1 = \frac{4h}{6}, \quad w_2 = \frac{h}{6}$$

If we want to use Simpson's rule for more finer partitions, we must first make sure that the number of intervals is even, say, $2M$ with $h = \frac{b-a}{2M}$. Then we can write

$$\begin{aligned}
 \int_a^b f(x) \, dx &= \sum_{k=1}^M \int_{x_{2k-2}}^{x_{2k}} f(x) \, dx \approx \sum_{k=1}^M \frac{h}{3} [f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})] \\
 &= \frac{h}{3} [f_0 + 4f_1 + 2f_2 + 4f_3 + 2f_4 + \cdots + 2f_{2M-2} + 4f_{2M-1} + f_{2M}] \\
 &= \frac{h}{3} (f_0 + f_{2M}) + \frac{2h}{3} \sum_{k=1}^{M-1} f_{2k} + \frac{4h}{3} \sum_{k=1}^M f_{2k-1}
 \end{aligned}$$

Composite Simpson's Rule

Because Simpson's rule captures part of the concavity of an underlying function, it is generally a better approximate than the trapezoid rule.

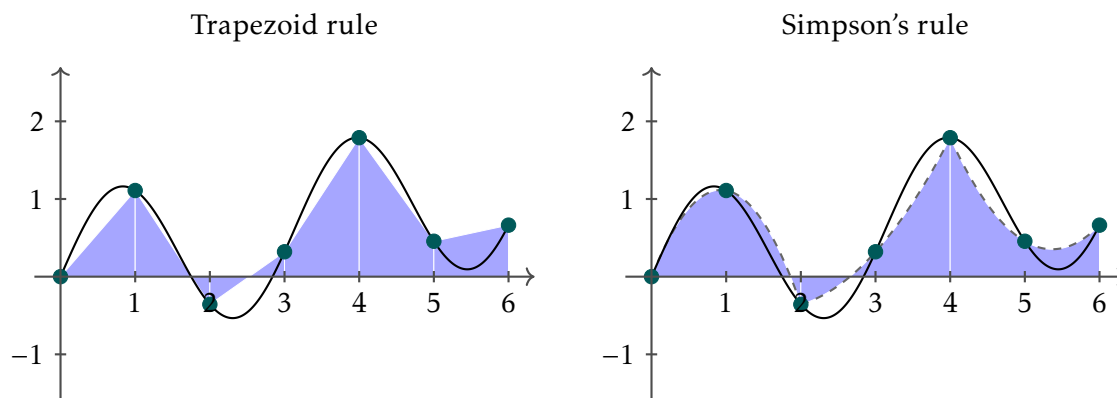


Figure 3.3: Trapezoid vs. Simpson's Rule on $[0, 6]$ with 6 equal subintervals for $f(x) = \sin(2x) + 0.2x$.

■ Question 38.

□

Write a code to approximate a definite integral using composite Simpson's rule.

Then, use your code to find the smallest number of equally-spaced subintervals that produces a composite Simpson's approximation of

$$\int_0^b x^2 \sin(2x) \, dx$$

within an error tolerance* of 0.001 in the following cases:

- (a) where b is the first zero of the integrand $x^2 \sin(2x)$ to the right of zero.
- (b) where b is the second zero of the integrand to the right of zero.

*Use Mathematica's NIntegrate command or Python's quad from scipy.integrate to get the "true" value of the integral.

Using Higher Degree Splines

Using Mathematica, the Newton-Cotes quadrature formula for degree 3 and 4 were computed to be as follows:

$$\boxed{n=3} \quad \int_{x_0}^{x_3} f(x) dx \approx \frac{3h}{8} [f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)] \quad (\text{“Simpson’s Three-Eighths rule”})$$

$$\boxed{n=4} \quad \int_{x_0}^{x_4} f(x) dx \approx \frac{2h}{45} [7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)]$$

Conclusion. This will increase the number of evaluations of f (often very costly), but hopefully will significantly decrease the error. This is essentially true until $n = 8$. After that point, negative weights appear in the Newton-Cotes formulas, and rounding error can become a significant feature of the calculations. For this reason, the Newton-Cotes formulas beyond $n = 7$ are rarely used.

Note: Here is another example to emphasize why high-degree Newton–Cotes rules can be a bad idea. Recall that Runge’s function $f(x) = \frac{1}{1+x^2}$ gave a nice example for which the polynomial interpolant at uniformly spaced points over $[-1, 1]$ fails to converge uniformly to f . This fact suggests that Newton–Cotes quadrature will also fail to converge as the degree of the interpolant grows, and indeed it does. On the other hand, integrating the interpolant at Chebyshev nodes, called *Clenshaw–Curtis quadrature*, does indeed converge.

3.1.4 Gaussian Quadrature

In general, we are interested in quadrature rules for *weighted* integrals of the type we saw last chapter,

$$I_w(f) := \int_a^b f(x) w(x) dx \approx \sum_{i=0}^n w_i f(x_i),$$

where $w(x)$ is a fixed nonnegative, continuous, integrable weight function on $[a, b]$ (typically $w(x) > 0$ a.e.).

The main motivation for weighted quadrature rules is to handle poorly behaved integrands – singular, discontinuous, highly oscillatory, and so on – where the “bad” behavior is known and can be factored out into $w(x)$.

For example, suppose need to estimate integrals of the form

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx, \quad \int_{-1}^1 f(x) e^{-x} dx,$$

or more generally $\int_a^b f(x) w(x) dx$ where w contains the dominant variation. By designing a quadrature rule with $w(x)$ taken into account, we can obtain fast convergence as long as the remaining factor $f(x)$ is

smooth, regardless of how “bad” $w(x)$ is. Moreover, the rule can be re-used for many different $f(x)$ as long as $w(x)$ remains the same.

Weighted integrals also arise naturally:

- when doing change of variables, since a Jacobian factor becomes a weight; and
- in probability and statistics, since expectations have the form $\mathbb{E}[f(X)] = \int f(x)\mu(x)dx$;

This is where **Gaussian quadrature** becomes useful. We will start with a family of orthogonal polynomials $\{\varphi_k\}_{k \geq 0}$ with respect to the inner product with weight function $w \in C[a, b]$.

$$\langle f, g \rangle = \int_a^b f(x)g(x)w(x)dx$$

Recall that φ_{n+1} has $(n+1)$ distinct roots x_i . We use these roots as the nodes to construct the Lagrange basis $\{\ell_i\}_{i=0}^n$ and define the quadrature weights by

$$w_i = \int_a^b \ell_i(x)w(x)dx$$

Then the resulting rule is

$$\int_a^b f(x)w(x)dx \approx \sum_{i=0}^n w_i f(x_i) = \sum_{i=0}^n \left(\int_a^b \ell_i(x)w(x)dx \right) f(x_i) =: Q_{G,n}(f),$$

where the nodes x_i are the roots of φ_{n+1} .

Note: When $w(x) = 1$ on $[-1, 1]$, the orthogonal polynomials are the Legendre polynomials. The corresponding Gaussian rule is the Gauss–Legendre quadrature rule.

One advantage of this complicated quadrature rule is that the weights are all positive (as opposed to higher order Newton-Cotes). But that’s really not enough to justify the complexity of the construction. So, why would anyone do this? To answer that, let’s talk about how we measure accuracy of quadratures rules.

§3.2 Degree of Exactness

Observe that if $f \in \mathcal{P}_n$, its polynomial interpolant $p_n \in \mathcal{P}_n$ is exactly f , and so the exact integral of p_n is the same thing as the exact integral of f . However, there are special cases where a degree- n interpolant will exactly integrate polynomials of higher degree. This motivates the next definition.

Definition 3.2.20

Fix a nonnegative, integrable weight function w on $[a, b]$ and define

$$I_w(f) := \int_a^b f(x)w(x)dx.$$

A quadrature rule

$$Q(f) := \sum_{j=0}^n w_j f(x_j)$$

has **degree of exactness** m if

$$Q(p) = I_w(p) \quad \text{for all } p \in \mathcal{P}_m.$$

Clearly a degree- n quadrature rule has degree of exactness at least n . Thus it is particularly interesting to see circumstances in which this degree of exactness is exceeded.

■ Question 39.

□

Check that although Simpson's rule is based on quadratic interpolants, it is exact for all cubic polynomials.

The next observation explains why degree of exactness is useful for error estimation. If Q is exact on $\mathcal{P}_k$, then subtracting any $p \in \mathcal{P}_k$ does not change the quadrature error:

$$Q(f) - I_w(f) = Q(f - p) - I_w(f - p).$$

This yields an a priori error bound in terms of best uniform approximation.

Theorem 3.2.21

Suppose w is nonnegative and integrable on $[a, b]$, and suppose the quadrature rule

$$Q(f) = \sum_{j=0}^n w_j f(x_j)$$

is exact for all polynomials in $\mathcal{P}_k$. Then for every $f \in C([a, b])$ and every $p \in \mathcal{P}_k$,

$$|Q(f) - I_w(f)| \leq \left(\sum_{j=0}^n |w_j| + \int_a^b w(x)dx \right) \|f - p\|_{\infty, [a, b]}.$$

Consequently the problem of error estimates for quadrature is reduced to the previously studied problem of best approximation.

$$|Q(f) - I_w(f)| \leq C \inf_{p \in \mathcal{P}_k} \|f - p\|_{\infty, [a, b]}, \quad C := \sum_{j=0}^n |w_j| + \int_a^b w(x)dx.$$

■ Question 40.

□